

# Gün 10 — Farklı Senaryolarda Tespit Başarımının Ölçülmesi

## Hedef

Görsel sıkıştırma ve farklı format kombinasyonlarında sistemin tespit başarımını ölçmek: en az 4 farklı test senaryosu üretip her senaryoda 1-2. haftanın üç analiz modülünü ( `detect_trailer` + `entropy` + `size_analysis` ) çalıştırmak, sonuçları tablo halinde raporlamak ve yanlış pozitif / yanlış negatif oranlarını not etmek.

## Yaklaşım

### 1. Modülleri birleştiren pipeline — `scripts/analyze.py`

1-2. hafta boyunca `detect_trailer.py` , `size_analysis.py` ve `entropy.py` birbirinden bağımsız CLI script'leri olarak yazılmıştı. Gün 10'da her test dosyası için üçünü ayrı ayrı çalıştırmak yerine, üçünü tek bir çağrıda birleştiren `scripts/analyze.py` eklendi:

```
from detect_trailer import analyze as detect_trailer_analyze
from entropy import DEFAULT_BLOCK_SIZE, compute_block_entropies
from size_analysis import analyze as size_analyze

def analyze(path, block_size=DEFAULT_BLOCK_SIZE) -> dict:
    trailer_result = detect_trailer_analyze(path)
    size_result = size_analyze(path)
    ent_result = entropy_summary(data, block_size, trailer_result["hidden_video_offset"])
    return {"polyglot_status": trailer_result["polyglot_status"],
            "trailer": trailer_result, "size": size_result, "entropy": ent_result}
```

Script hiçbir sinyali tek başına "kesin karar" olarak kullanmıyor; üç modülün sonucunu da yan yana raporluyor. Sinyallerin ağırlıklı bir tehdit skoruna ( `threat_score` ) dönüştürülmesi bilinçli olarak Gün 14'e (API katmanı) bırakıldı — Gün 10'un amacı tespit *başarımını ölçmek*, henüz nihai bir skorlama formülü tasarlamak değil.

`entropy_summary()` , `detect_trailer` 'ın bulduğu `hidden_video_offset` 'i sınır noktası olarak kullanıp bu noktadan önceki ve sonraki blokların ortalama entropy'sini ayrı ayrı hesaplıyor ( `entropy_delta` ); sınır bulunamazsa (temiz dosya veya trailer yok) yalnızca dosyanın genel ortalama entropy'si raporlanıyor.

### 2. Senaryo dosyalarının üretimi — `scripts/make_test_scenarios.py`

Plandaki 4 kategoriden (1) PNG+MP4 ve (2) JPEG+MP4 polyglot'lar Gün 3'te zaten mevcuttu ( `samples/polyglot_png.png` , `samples/polyglot_jpg.jpg` ). (3) "yeniden sıkıştırılmış/optimize edilmiş polyglot" senaryosu için iki farklı, gerçekçi alt-durum üretildi:

- **3a — embed sonrası yeniden sıkıştırma:** var olan bir polyglot dosya PIL ile açılıp ( `Image.open().load()` ) tekrar kaydediliyor ( `img.save(..., optimize=True)` ). PIL yalnızca kendi dekode ettiği piksel verisini yazdığı için, EOF sonrasındaki gizli video baytları bu işlemde tamamen kayboluyor. Bu, bir platformun (ör. bir sosyal medya sunucusunun) yüklenen görseli kendi pipeline'ında yeniden encode etmesini simüle ediyor.
- **3b — embed öncesi yeniden sıkıştırma:** taşıyıcı görsel önce farklı bir sıkıştırma seviyesiyle ( `compress_level=9` / `quality=50` ) kaydediliyor, video bu *yeni* taşıyıcının arkasına ekleniyor. Bu, tespitin taşıyıcının iç sıkıştırma detaylarından ne kadar bağımsız çalıştığını ölçüyor.

(4) "temiz görseller" kategorisine Gün 1'in `sample.png / sample.jpg` dosyalarına ek olarak Gün 7'nin LSB steganografi örneği ( `lsb_stego_sample.png` — gizli veri var ama trailer-append **değil**) ve sentetik bir 128×128 gradient ( `clean_gradient.png` ) eklendi; amaç sadece "boş" dosyalarla değil, "başka türden gizli veri içeren ama bizim tehdit modelimize (trailer-append) girmeyen" bir dosyayla da false-positive oranını sınamak.

Üretilen dosyalar `samples/test_matrix/` altına yazılıyor ve `samples/extracted/` ile aynı mantıkla `.gitignore` 'a eklendi (script her çalıştırıldığında yeniden üretilebilir, git'e dahil edilmesi gerekmiyor).

## Script'ler

```
python scripts/make_test_scenarios.py
python scripts/analyze.py --file <dosya>
python scripts/analyze.py --file <dosya> --json
```

## Test Sonuçları

10 dosya (4 kategori) üzerinde `analyze.py` çalıştırıldı. Tam sonuç tablosu ve JSON alan bazlı ayrıntılar `docs/test-sonuclari.md` içinde; özet aşağıda:

| #  | Dosya                                  | Gerçek durum | trailer.polyglot_status | size.suspicious (sapma) | Trailer sonucu                               |
|----|----------------------------------------|--------------|-------------------------|-------------------------|----------------------------------------------|
| 1  | <code>polyglot_png.png</code>          | polyglot     | <b>EVET</b>             | EVET (+49.3%)           | ✓<br>doğru                                   |
| 2  | <code>polyglot_jpg.jpg</code>          | polyglot     | <b>EVET</b>             | EVET (+294.0%)          | ✓<br>doğru                                   |
| 3a | <code>recompressed_post_png.png</code> | temiz*       | hayır                   | hayır (−92.8%)          | ✓<br>doğru                                   |
| 3a | <code>recompressed_post_jpg.jpg</code> | temiz*       | hayır                   | hayır (−51.9%)          | ✓<br>doğru                                   |
| 3b | <code>recompressed_pre_png.png</code>  | polyglot     | <b>EVET</b>             | EVET (+47.7%)           | ✓<br>doğru                                   |
| 3b | <code>recompressed_pre_jpg.jpg</code>  | polyglot     | <b>EVET</b>             | EVET (+256.4%)          | ✓<br>doğru                                   |
| 4  | <code>sample.png</code>                | temiz        | hayır                   | hayır (−91.2%)          | ✓<br>doğru                                   |
| 4  | <code>sample.jpg</code>                | temiz        | hayır                   | hayır (+11.6%)          | ✓<br>doğru                                   |
| 4  | <code>lsb_stego_sample.png</code>      | temiz        | hayır                   | <b>EVET (+45.6%)</b> ⚠  | ✓<br>doğru (trailer)<br>/ ✗<br>yanlış (size) |
| 4  | <code>clean_gradient.png</code>        | temiz        | hayır                   | hayır (−96.3%)          | ✓<br>doğru                                   |

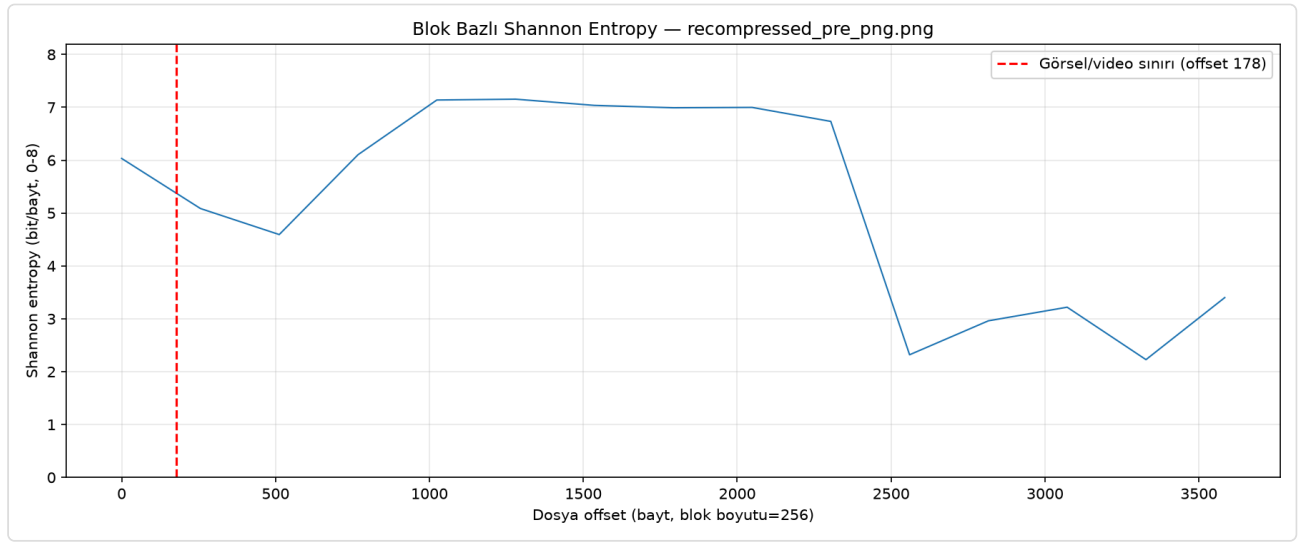
\* 3a'da zemin gerçeği "temiz": recompression videoyu fiilen siliyor (bkz. aşağıdaki bulgu #2).

### Yanlış Pozitif / Yanlış Negatif Oranları

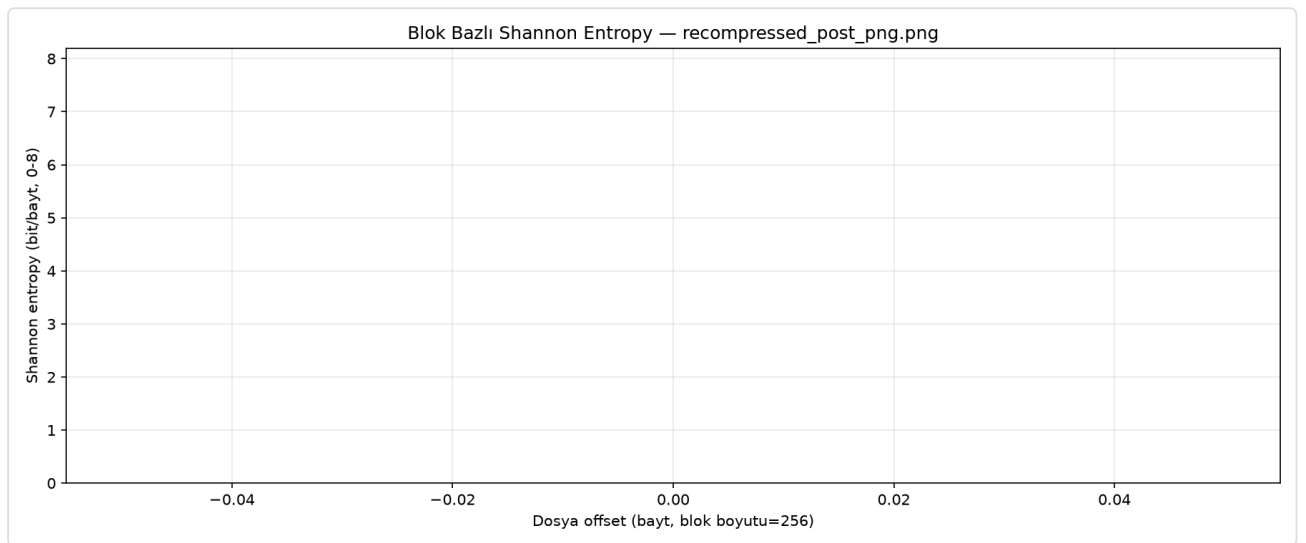
| Sinyal                              | Temiz dosyalarda FP                  | Polyglot dosyalarda FN |
|-------------------------------------|--------------------------------------|------------------------|
| trailer.polyglot_status (ana karar) | 0/6 = %0                             | 0/4 = %0               |
| size.suspicious (tek başına)        | 1/6 = %16.7 ( lsb_stego_sample.png ) | 0/4 = %0               |

### Görsel doğrulama — entropy grafikleri

Taşıyıcı embed öncesi farklı sıkıştırımla üretilmiş olsa bile (senaryo 3b), görsel/video sınırı entropy grafiğinde hâlâ net görünüyor:



Embed *sonrası* yeniden sıkıştırılmış dosyada ise (senaryo 3a) trailer tamamen silindiği için sınır çizgisi hiç yok — dosya tek blokludur, düz bir grafik:



### Öne çıkan bulgular

1. **Ana tespit sinyali ( detect\_trailer ) 10/10 dosyada doğru: %0 FP, %0 FN.** Taşıyıcının embed öncesi farklı sıkıştırma seviyesiyle kaydedilmesi tespiti hiç etkilemiyor — trailer taraması EOF sonrası baytlara baktığı için taşıyıcının iç kodlama detaylarından bağımsız.
2. **Embed sonrası yeniden sıkıştırma trailer'ı tamamen yok ediyor** ( recompressed\_post\_png.png : 3688 → 178 bayt). Sistem bunu doğru şekilde "temiz" raporluyor; bu bir tespit hatası değil, çünkü gizli video verisi bu noktada adli olarak da gerçekten kurtarılamaz hale geliyor — planın Gün 10 risk notuyla birebir örtüşüyor. Pratik sonucu: bir platformun sunucu tarafı görsel işleme pipeline'ı bu tip polyglot'ları kendiliğinden etkisiz hale getiriyor.
3. **size\_analysis tek başına çok daha gürültülü bir sinyal.** lsb\_stego\_sample.png 'de trailer yok ama LSB steganografi PNG'nin gerçek boyutunu Gün 6'nın basit "~%20 sıkıştırma" varsayımının ötesine taşıdığı için sapma %45.6 çıkıyor ve dosya yanlışlıkla "şüpheli" işaretleniyor. JPEG polyglot'larda da sapma çok değişken (+%294, +%256) — Gün 6 risk notuyla uyumlu.
4. **entropy.entropy\_delta bağımsız bir tespit sinyali değil, detect\_trailer 'ı doğrulayan tamamlayıcı bir sinyal:** yalnızca trailer bir sınır bulduğunda hesaplanabiliyor, ama bulunduğu tüm polyglot dosyalarında görsel bölgesi (~6.0-6.4 bit/bayt) ile video bölgesi (~5.1-5.2 bit/bayt) arasında tutarlı ve belirgin bir düşüş gösteriyor.

## Kabul Kriterleri — Durum

- [x] docs/test-sonuclari.md en az 4 senaryo için sonuç satırı içeriyor (4 kategori, 10 dosya)
- [x] Sistem temiz dosyalarda %0'a yakın false-positive veriyor (ana trailer sinyali: 6 temiz dosyada tam olarak %0 FP)

## Notlar / Riskler

Planın öngördüğü risk gerçekleşti: "Görsel optimizasyon araçları (örn. PNG yeniden sıkıştırma) bazen trailer verisini bozabilir" — senaryo 3a bu durumu doğruluyor. Bulgu #2'de açıklandığı gibi bu bir sınırlama olarak kabul edilmeli (adli olarak veri kurtarılamıyor), ama aynı zamanda istenmeyen yükün kendiliğinden etkisizleşmesi bakımından olumlu bir yan etki de taşıyor. Ayrıca size\_analysis 'in LSB-stego gibi trailer içermeyen gizli veri türlerinde yanlış pozitif verebildiği görüldü; bu nedenle bu sinyal API katmanında (Gün 14) asla tek başına karar mercii olarak kullanılmamalı, yalnızca detect\_trailer sonucunu destekleyen ikincil bir gösterge olarak ağırlıklandırılmalı.